CAPRM-Flood: A Reproducible C++/Python Framework for Property-Level Flood Exposure Indexing

Sterling Alexander — M.S. Computer Science, RIT — Proposed Completion: August 2026

Summary

This capstone proposes **CAPRM-Flood**, a reproducible C++/Python geospatial framework for computing a **Relative Flood Exposure Index** for residential properties in a selected study region. The initial case study will target Monroe County / Rochester, NY, using publicly available property-location, flood-hazard, hydrography, elevation, precipitation, and administrative-boundary data.

For each property, the system will derive a constrained set of public-data features: FEMA flood-zone membership, distance to mapped water bodies, elevation or relative-elevation proxies, extreme-precipitation indicators, and optional structure/consequence proxies where credible data is available. These features will be normalized into interpretable component scores and combined into a transparent 0–100 index ranking properties by relative flood exposure within the study area.

The project will not attempt to predict actual flood losses, insurance premiums, property prices, or mortgage risk. The main computer science contribution is the design and evaluation of a reproducible high-performance geospatial processing pipeline, including a C++ spatial-lookup core benchmarked against standard Python geospatial tools for correctness, throughput, memory behavior, and scalability.

Problem and Scope

Existing flood-risk tools can be useful, but many are proprietary, difficult to reproduce, or focused on consumer-facing outputs rather than transparent computational methodology. This project focuses on a narrower CS problem: given residential property locations and public flood-related geospatial datasets, can a system compute transparent property-level flood exposure scores efficiently, reproducibly, and at scale?

The output is a relative index, not a supervised prediction of observed damage. It answers: relative to other properties in the selected region, how exposed does this property appear to flood-related physical risk based on public geospatial variables?

Practical System Behavior

CAPRM-Flood will run as a batch-processing pipeline and optional FastAPI service. A user will provide a table of property locations containing at minimum a property identifier and coordinates. For each property, the system will determine whether the location intersects a FEMA flood-hazard zone, compute distance to the nearest mapped water feature, extract elevation or terrain-derived values, attach a precipitation-frequency indicator, normalize each feature into a component score, and combine the components into a 0–100 Relative Flood Exposure Index.

The system output will be a CSV or JSON report containing each property's component scores, composite index, regional percentile rank, and evidence fields used to compute the score.

Related Tools and Differentiation

Several existing tools address adjacent problems. First Street's Flood Factor provides property-specific flood-risk information based on flood likelihood, projected flood depth, and cumulative 30-year risk, including rainfall-driven, riverine, coastal, and storm-surge mechanisms [1, 2]. FEMA's National Risk Index provides baseline natural-hazard risk measures using expected annual loss, social vulnerability, and community resilience at Census tract and county scales [3]. FEMA Hazus provides standardized tools and data for estimating losses from floods and other hazards, while FEMA's Flood Assessment Structure Tool (FAST) calculates building-level flood impacts using building and flood-depth inputs [4, 5]. Standard Python geospatial tools such as GeoPandas/Shapely already support spatial joins and spatial indexing for point-in-polygon and related spatial operations [6, 7].

CAPRM-Flood is not intended to replace these tools or claim actuarial-grade loss prediction. Its contribution is narrower: an open, reproducible C++/Python geospatial computation pipeline for property-level flood exposure indexing, with explicit variables, transparent scoring assumptions, correctness comparison against Python geospatial baselines, and performance benchmarks for spatial lookup, throughput, memory behavior, and scaling.

Specific Inputs and Data Sources

The minimum required input will be a property table containing `property_id`, `latitude`, and `longitude`. Optional fields may include assessed value, building size, structure type, year built, parcel area, or land-use code if credible parcel or assessment attributes are available. The primary property-location input will be NYS Tax Parcel Centroid Points for Monroe County. If parcel filtering, licensing, or attribute access becomes problematic, the system will use synthetic residential property points within the study region as a reproducible fallback.

Property locations	NYS Tax Parcel Centroid Points for Monroe County; required fields: property ID and coordinates.
Flood-zone exposure	FEMA National Flood Hazard Layer / FEMA Map Service Center data; extracted feature: flood-zone membership or Special Flood Hazard Area status.
Water proximity	USGS National Hydrography Dataset, NHDPlus HR, or 3D Hydrography Program data; extracted feature: distance to nearest mapped river, stream, lake, or drainage feature.
Elevation / terrain	USGS 3D Elevation Program / The National Map elevation data; extracted features: elevation, local relative elevation, and optional slope.
Extreme precipitation	NOAA Atlas 14 / Precipitation Frequency Data Server; extracted feature: precipitation-frequency or rainfall-intensity indicator, such as 24-hour 100-year precipitation.
Regional context	U.S. Census TIGER/Line boundaries; extracted features: county, tract, municipality, or regional grouping for ranking and reporting.

Feature Set and Index

CAPRM-Flood will compute interpretable component scores from the extracted geospatial features.

Flood-zone component	Higher score when a property intersects a mapped high-risk flood zone or equivalent hazard area.
Water-proximity component	Higher score when a property is closer to mapped rivers, streams, lakes, coastlines, or drainage features.
Elevation / terrain component	Higher score for lower elevation, low relative elevation, unfavorable slope, or equivalent terrain-derived proxy.
Precipitation component	Higher score for locations with stronger heavy-precipitation or rainfall-intensity indicators.
Structure / consequence component	Optional score based on assessed value, building size, structure type, or synthetic placeholder if credible data is unavailable.

$$\text{Flood Exposure Index} = w_z C_z + w_w C_w + w_e C_e + w_p C_p + w_s C_s$$

Each component will be normalized to a common scale. Weights will be explicit and configurable; sensitivity analysis will evaluate how rankings change under different assumptions. The precipitation component will be treated as a rainfall-driven flood-hazard proxy, not as a full hydrologic simulation.

Example Output

For each property, CAPRM-Flood will produce an output record similar to:

Property ID	10482
Flood-zone component	90/100; property intersects mapped FEMA high-risk flood zone.
Water-proximity component	75/100; property is 140 meters from nearest mapped stream feature.
Elevation / terrain component	82/100; property has low relative elevation within local area.
Precipitation component	70/100; property is in a high rainfall-intensity percentile region.
Optional structure component	60/100; credible structure/consequence proxy available.
Composite index	81/100.
Regional rank	93rd percentile among properties in the study region.

Technical Contribution

Python will handle data ingestion, cleaning, coordinate normalization, orchestration, scoring, reporting, experiment configuration, and FastAPI access. C++ will handle the performance-critical spatial-processing stage: batch point-in-polygon lookup, property-to-flood-zone membership, nearest-water distance calculation, precipitation-grid or regional precipitation lookup, spatial indexing, and large-batch feature extraction.

The CS contribution is not a new flood-risk theory; it is the implementation and evaluation of a reproducible, high-performance geospatial computation system:

- modular C++/Python geospatial processing architecture;
- C++ spatial-lookup and feature-extraction core;
- correctness comparison against GeoPandas/Shapely or equivalent baselines;
- benchmarks for runtime, throughput, memory behavior, and scaling;
- Dockerized reproducibility, automated tests, and documented assumptions/limitations.

Evaluation Plan

Correctness: Compare C++ spatial-processing results against trusted Python geospatial outputs, including point-in-polygon agreement, nearest-distance agreement within tolerance, precipitation-grid or regional lookup agreement, reproducibility, and boundary-case behavior.

Performance: Benchmark the C++ spatial core against standard Python geospatial baselines across increasing dataset sizes, e.g., 1K, 10K, 100K, and potentially 1M property records. Metrics may include runtime, throughput, memory usage, preprocessing overhead, and scaling behavior.

Sensitivity / Case Study: Evaluate how the index changes under different weighting assumptions, including flood-zone-heavy, water-proximity-heavy, elevation-heavy, precipitation-heavy, and equal-weight configurations. Demonstrate the system on Monroe County / Rochester, NY with score distributions, component-score breakdowns, regional ranks, limitations, and uncertainty discussion.

Deliverables and Scope Controls

Expected deliverables: working C++/Python geospatial system; Dockerized execution environment; C++ spatial-lookup core; Python orchestration/scoring modules; CSV/JSON output reports; correctness tests; performance benchmarks; sensitivity analysis; FastAPI endpoint; documentation; final capstone report.

Scope exclusions: actual flood-loss prediction, insurance-premium estimation, property-price prediction, mortgage/credit modeling, replacement of FEMA/Hazus/proprietary products, full hydrologic simulation, all-hazard climate modeling, national deployment, and complex consumer dashboard development.

References

[1] First Street. "How is my Flood Factor calculated?" Accessed May 2026. `https://help.firststreet.org/hc/en-us/articles/360047585694-How-is-my-Flood-Factor-calculated`

[2] First Street. "Flood Model Methodology: Calculating property-level risk." Accessed May 2026. `https://help.firststreet.org/hc/en-us/articles/1500000359741-Flood-Model-Methodology-Calculating-property-level-risk`

[3] Federal Emergency Management Agency. "National Risk Index for Natural Hazards." Accessed May 2026. `https://www.fema.gov/flood-maps/products-tools/national-risk-index`

[4] Federal Emergency Management Agency. "Hazus." Accessed May 2026. `https://www.fema.gov/flood-maps/products-tools/hazus`

[5] Federal Emergency Management Agency. "Flood Assessment Structure Tool (FAST)." Accessed May 2026. `https://msc.fema.gov/portal/resources/hazus`

[6] GeoPandas. "Spatial Joins." Accessed May 2026. `https://geopandas.org/en/stable/gallery/spatial_joins.html`

[7] GeoPandas. "Spatial indexing." Accessed May 2026. `https://geopandas.org/en/latest/docs/user_guide/spatial_indexing.html`

[8] New York State GIS Program Office. "NYS Parcels." Accessed May 2026. `https://gis.ny.gov/parcels`

[9] Federal Emergency Management Agency. "National Flood Hazard Layer." Accessed May 2026. `https://www.fema.gov/flood-maps/national-flood-hazard-layer`

[10] U.S. Geological Survey. "National Hydrography Dataset." Accessed May 2026. `https://www.usgs.gov/national-hydrography/national-hydrography-dataset`

[11] U.S. Geological Survey. "The National Map Data Download and Visualization Services." Accessed May 2026. `https://www.usgs.gov/the-national-map-data-delivery/gis-data-download`

[12] NOAA National Weather Service. "Precipitation Frequency Data Server." Accessed May 2026. `https://hdsc.nws.noaa.gov/pfds/`

[13] U.S. Census Bureau. "TIGER/Line Shapefiles." Accessed May 2026. `https://www.census.gov/geographies/mapping-files/time-series/geo/tiger-line-file.html`